本地代码安全审核

方法选择、检查能力、人工判断与整改闭环

从“发现线索”走向“有依据的判断”

围绕指定代码版本，开展规则检查、重点代码审查和必要核验，形成可解释、可整改、可追溯的安全审核结果。

固定版本

明确代码范围与比较基线

人工判断

核查调用链、权限与现有防护

闭环处置

出具报告并核验修复效果

先明确检查范围

审核结论对应指定版本与实际覆盖范围

01

代码版本

仓库、分支与完整 Commit

增量检查的比较基线

明确是否包含未提交内容

02

检查对象

源码、JSP、配置文件

已有 JAR 与可识别依赖

接口、权限及敏感业务模块

03

范围确认

第三方库、测试和生成代码

子模块及特殊归档处理

规则排除项与人工抽查范围

全量检查覆盖约定范围内受支持的文件，不等于人工逐行阅读全部代码。

整体实施流程

工具扫描、工具审核、工具复审与人工确认分别开展

01
获取与固定代码
确认范围和审核版本

02
工具扫描
全量或增量发现线索

03
工具审核（初审）
分析证据与风险成立条件

04
工具复审
独立判断并检查初审依据

05
人工代码审核
抽查源码、追踪调用链

06
人工审定报告
核查确认状态与整改建议

07
研发整改
提交修复版本和说明

08
人工修复核验
复查根因及关联路径

顺序：01 → 02 → 03 → 04 → 05 → 06 → 07 → 08；核验未通过，返回整改。

工具审核与复审分别保存建议和依据；一致性检查不能替代人工判断，分歧及证据不足项转人工处理。

全量与增量如何配合

首次建立基线，后续关注变化及其影响

FULL

全量检查

适用：首次审核、重大上线、系统重构。

检查指定版本的约定范围，梳理高风险入口、权限与敏感操作，形成初始问题清单。

DELTA

增量检查

适用：日常迭代、分支合入、整改复查。

对比当前版本和指定基线，识别新增、存量及变化线索，并人工分析变更的上下游影响。

分支检查可使用共同祖先作为基线；比较状态“已修复”仍需结合人工核验确认。

五类开源工具，覆盖不同对象

规则命中、组件 CVE 和编码规范问题需要分别解读

辅助检查工具	检查对象	主要能力与边界
Semgrep	源码 / JSP / JSPX	通用代码规则与专项 JSP 规则包；覆盖依赖所选规则。
Gitleaks	目录与扫描快照	疑似硬编码凭据；不代表检查完整 Git 提交历史。
Trivy	依赖组件与文件	已知组件漏洞及密钥；识别依赖于组件元数据和本地库。
SpotBugs / FindSecBugs	Java 字节码	利用已有 JAR 分析业务类；需要正确的业务包范围。
Alibaba P3C-PMD	Java 源文件	阿里编码规范检查；规范问题不直接等同安全漏洞。

老项目按源码、JSP、字节码和依赖分层检查；缺少源文件时，应明确未覆盖范围。

JSP / JSPX 专项扫描规则

针对老项目补充本地规则包，识别请求参数进入敏感操作的线索

CWE-22

路径穿越 / 文件访问

检查请求参数是否参与文件路径、文件流构造或图片写入。
人工核查：根目录限制、路径规范化、文件名白名单及访问权限。

CWE-78

系统命令执行

检查请求参数是否进入 Runtime.exec 或 ProcessBuilder 等命令构造。
人工核查：输入可控性、命令及参数白名单、可达路径。

CWE-601

开放重定向

检查请求参数是否控制 response.sendRedirect 的跳转目标。
人工核查：是否仅允许预定义站内路径，是否存在绕过方式。

当前为静态模式规则，包含直接传参与部分变量转接模式；不代表完整 JSP 编译、跨文件数据流或全部风险覆盖。

前端专项规则与检查范围

独立前端规则包，补充 JavaScript、TypeScript 及组件源码检查

JS / TS / JSX / TSX

浏览器风险模式

检查通配 postMessage、浏览器输入控制跳转、敏感凭据写入 localStorage 及明文 HTTP 请求。React 源码复用这些规则。

Vue 单文件组件

限定模式补充

检查路由参数直接进入 v-html、直接控制跳转，以及跨窗口消息和敏感凭据存储。结合完整源码人工排除误报。

规则使用与边界

覆盖按约定执行

使用前端专项规则与正反例验证。压缩 JS 与部分第三方脚本按范围排除；不重新启用 eval 和直接 innerHTML 规则。

JS/TS 采用语法及局部数据流规则，Vue 采用限定文本模式；不代表完整框架语义或跨文件分析。

工具审核与工具复审

先分析风险线索，再复查依据，分别记录两次判断

第一步

工具审核（初审）

结合命中位置、源码上下文或依赖信息，分析风险条件与现有防护，输出初审建议、依据及可信度。

第二步

工具复审

再次检查证据，并复核初审依据是否充分。分别记录复审建议、依据及可信度，避免只重复初审结论。

第三步

一致性与人工处理

核对两次判断是否一致、证据是否完整。分歧、证据不足或执行失败项交由人工补充核查；工具结果不等于人工确认。

同依赖文件、组件及版本的 CVE 可按组审核与复审，保留各条明细；工具可信度不是漏洞成立概率。

人工审核具体做什么

从代码证据出发，判断风险是否成立及整改优先级

源码抽查与入口梳理

检查高风险接口、文件处理、认证授权和敏感操作。

调用链与输入追踪

追踪输入来源、参数流转和实际可达路径。

权限与防护核查

确认触发身份、校验、编码、白名单与隔离措施。

风险确认与建议

结合影响判断风险或误报，记录依据并提出修复建议。

对工具存在分歧、证据不足或执行失败的条目，补充上下文后判断，不将其直接认定为漏洞。

如何去重与组织整改

减少重复处理，同时保留每项问题的依据

候选结果

先判断，再筛选

人工确认风险、误报或待验证。
保留判断依据，避免将所有工具命中直接列为已确认漏洞。

归并原则

同根因，合理归组

同一位置的重复线索去重。
相同依赖文件、组件及版本的多个 CVE，可统一组织升级整改。

交付表达

明细与统计分开

保留成员 CVE、位置和证据。
整改组数与漏洞条目数分别标识；不同文件或版本分别处理。

示例：同一 JAR 的 6 个 CVE 可归为 1 个整改组；6 条漏洞明细不能因此被省略。

审核工作如何体现为交付成果

用版本、范围、证据与结论说明实际完成的工作

获取与梳理	代码版本记录、模块与技术栈梳理、审核范围
工具检查与复核	通用与 JSP 专项规则、扫描状态、工具初审及复审依据
人工审核	实际抽查模块、调用链判断、问题确认与误报依据
整改与核验	审核报告、整改清单、修复版本及核验结论

具体仓库数、代码量、问题数与已完成核验数量，以项目记录为准；本讲解材料不预设工作量。

漏洞库更新与检查时效

漏洞数据需要维护；新数据不会自动改变历史审核结论

检查之前

准备并更新

提前维护漏洞库和 Java 索引库，检查库是否可用，记录最近成功更新时间与维护结果。

检查期间

读取本地库

扫描阶段读取已准备的本地缓存，避免临时下载挤占检查时间。缺库或库过旧时先处理或明确限制。

持续维护

建议定时更新

建议每天在非检查时段更新；定时调度仍属待实施维护方案。需考虑错峰、失败通知和缓存保护。

使用新漏洞库评估代码时，应重新执行检查；更新库本身不会自动重扫或改写历史报告。

整改后，如何确认真正修复

规则不再命中，是参考信号，不是唯一验收依据

研发提交

修复版本与说明

提供修复 Commit、改动范围及自测结果。依赖升级需核查真实生效版本和兼容性，避免只修改声明而保留旧包。

审核核验

复查根因与绕过

检查修改是否消除根因，关联接口是否仍可绕过，是否引入新风险。核验结论分为通过、未通过和待验证。

暂不修复时，由责任方明确风险接受、补偿措施与复审期限；接受风险不等于已修复。

案例：一个依赖漏洞如何完成闭环

教学示例，不代表某个项目已经完成的审核结果

发现与工具复核	识别某 JAR 的多个 CVE，经工具初审、复审形成建议与依据。
人工判断	核对真实版本、使用方式、触发条件和修复版本。
整改	将同组件多个 CVE 组织为升级整改组，保留各条明细。
核验	检查升级是否生效、旧包是否移除及兼容性测试结果。

交付报告区分人工已确认风险与待人工确认项；需要只交付已确认漏洞时，应完成相应确认。